

CSC2161 PROGRAMMING IN C++

Dieudonne U



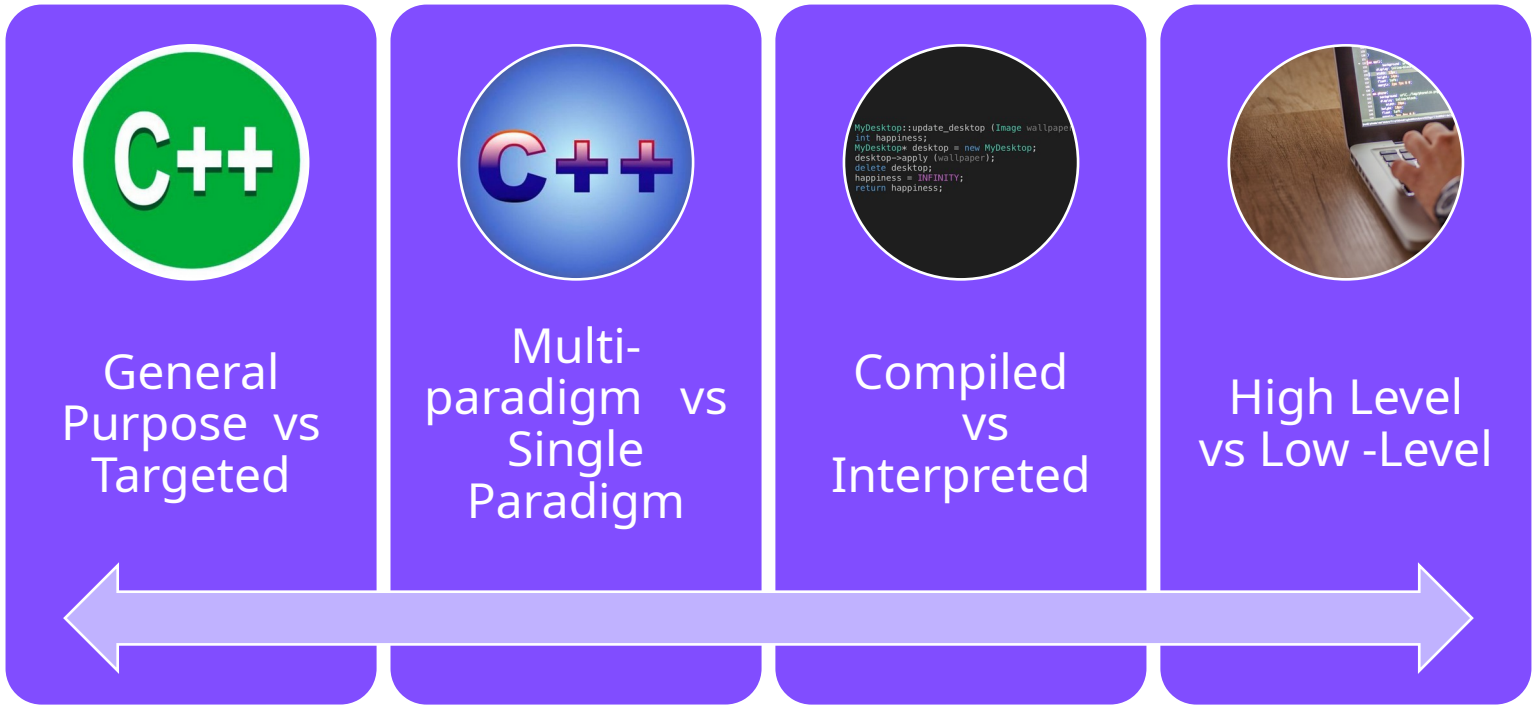
AGENDA

C++ History

Variables, Operators, Expressions

Control Structures

C++ Programming Language





General Purpose

- Used to develop applications in various domains: Finance, Telecommunication, Games.
- As opposed to targeted programming platforms such as MATLAB used in Engineering and Science Applications, R used in Data Science Applications..



This Photo by Unknown Author is licensed under CC BY-SA

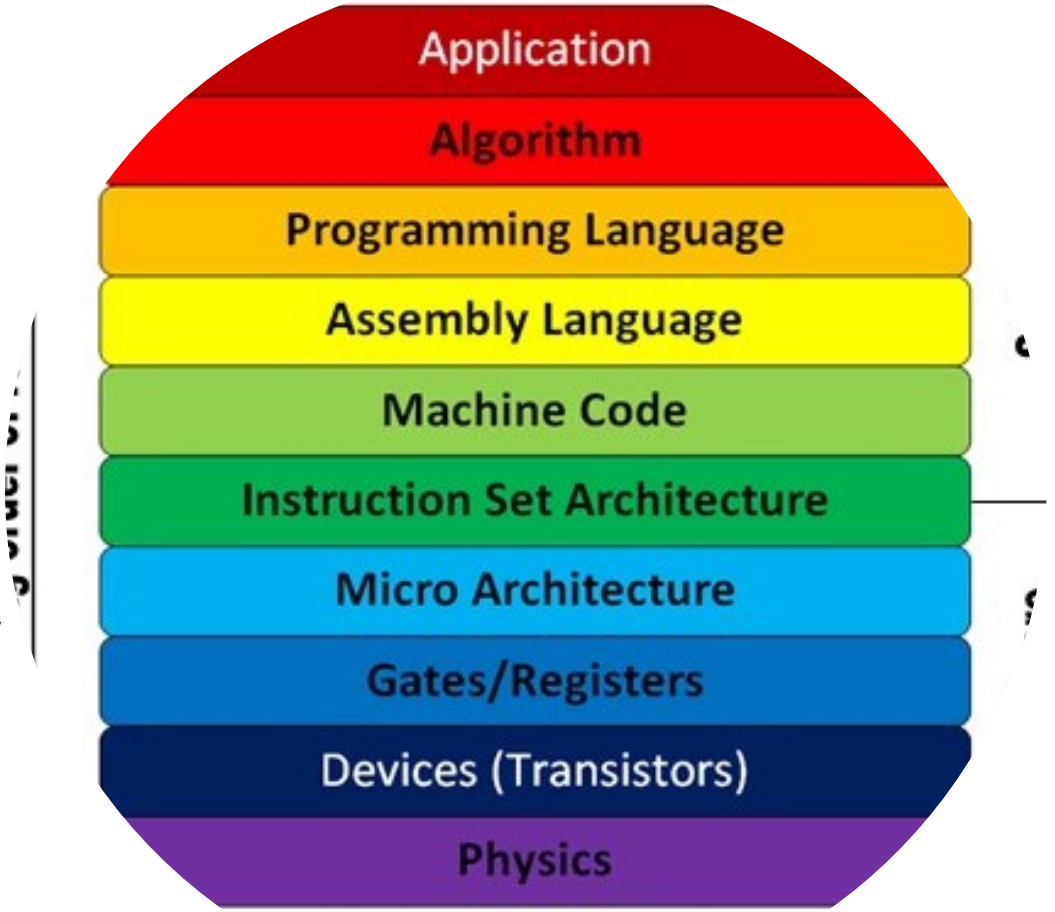
MULTI -PARADIGM

- A programming paradigm is a methodology to develop computer applications.
- There are many programming paradigms: Structured (Fortran, C), Functional (Haskell, Scala, Lisp), Object-oriented (Java, Python, C#), Logic (Prolog)
- C++ supports many programming paradigms : Structured, Object-Oriented, Generic Programming



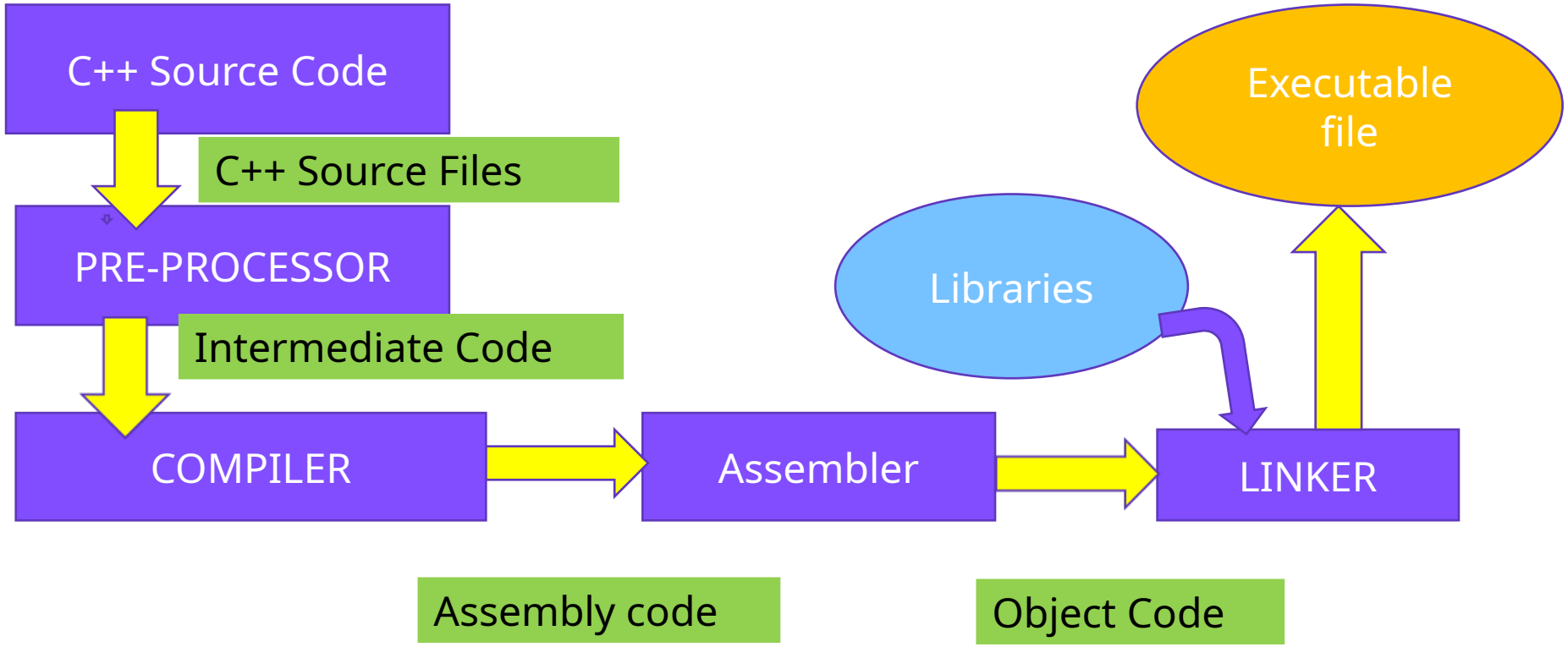
HIGH LEVEL

- Machine code : Manipulated instructions in binary format, specific to each machine architecture – No abstraction
- Assembly language: Uses commands to manipulate the content of registers. Converted in machine code by and assembler. Very low abstraction
- High Level programing languages: programs are written in a language easier to understand by human, but not understood by the CPU. Has to be either compiled or interpreted

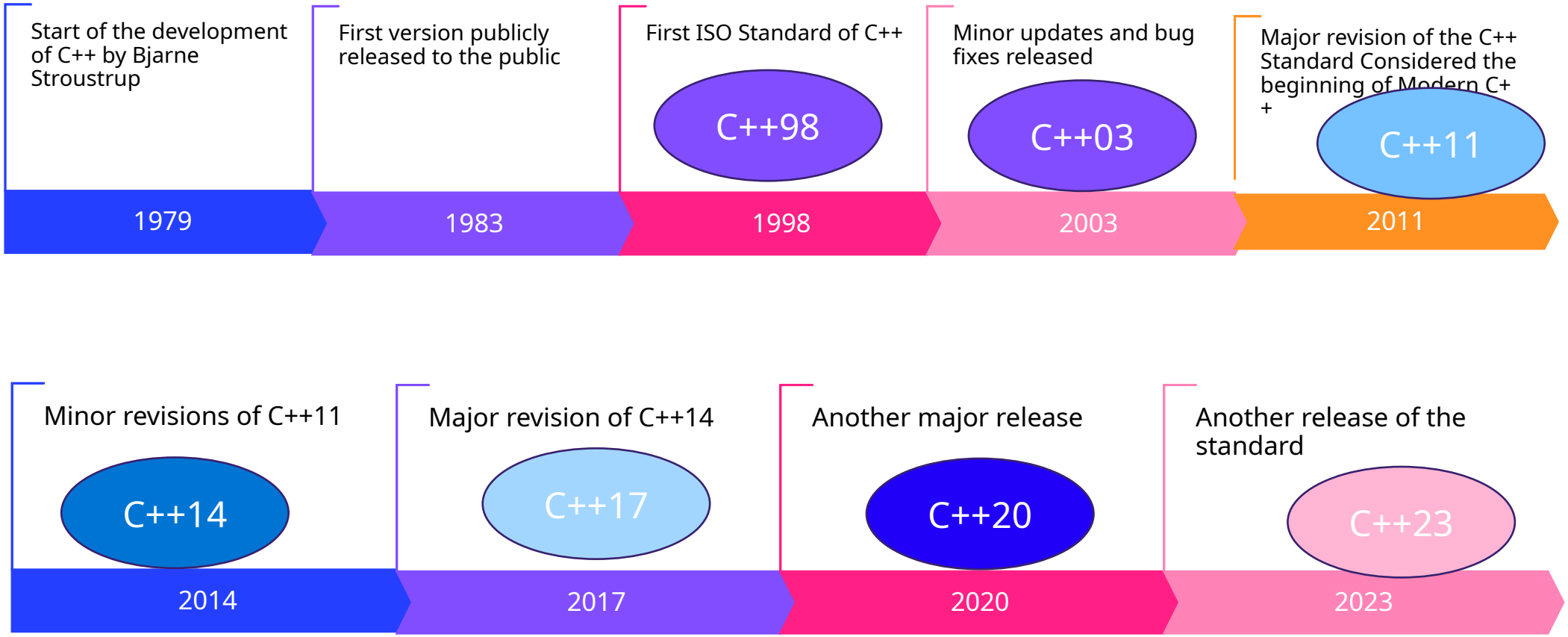


Layers of Abstraction

C++ Build Process



History of C++

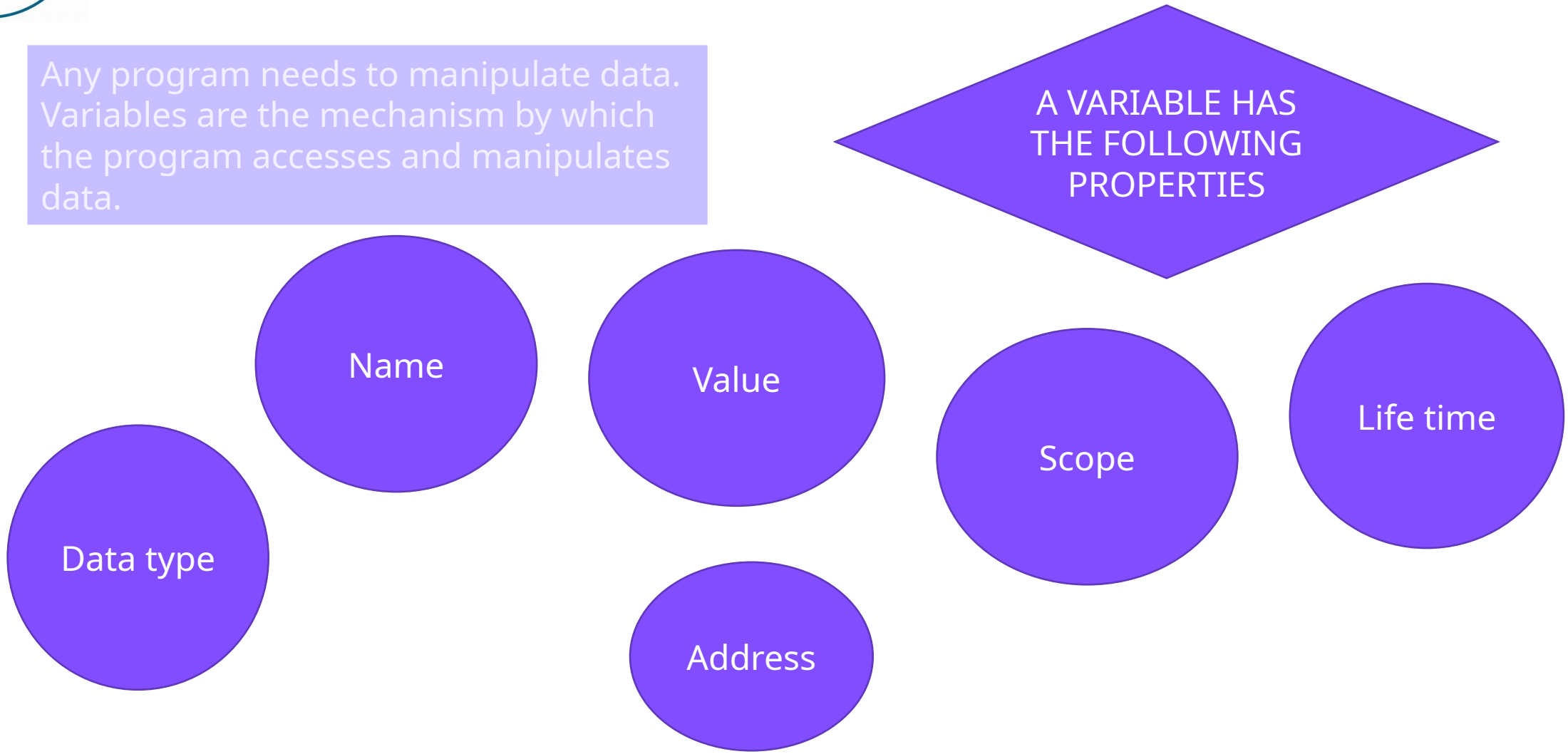

Start of the development of C++ by Bjarne Stroustrup
1979
First version publicly released to the public
1983
First ISO Standard of C++
C++98
1998
Minor updates and bug fixes released
C++03
2003
Major revision of the C++ Standard Considered the beginning of Modern C++
C++11
2011
Minor revisions of C++11
C++14
2014
Major revision of C++14
C++17
2017
Another major release
C++20
2020
Another release of the standard
C++23
2023



VARIABLES

A variable is a memory location destined to hold a value

Any program needs to manipulate data. Variables are the mechanism by which the program accesses and manipulates data.



A DATA TYPE SPECIFIES THE RANGE OF VALUES A VARIABLE CAN HAVE AND THE OPERATIONS IT CAN BE INVOLVED IN.

Built-in data types are defined by the language

- Integers: (whole numbers)short, int , long, long long.
- Real numbers: (Decimal point numbers) float, double
- A Single character: char
- Boolean (true, false) : bool

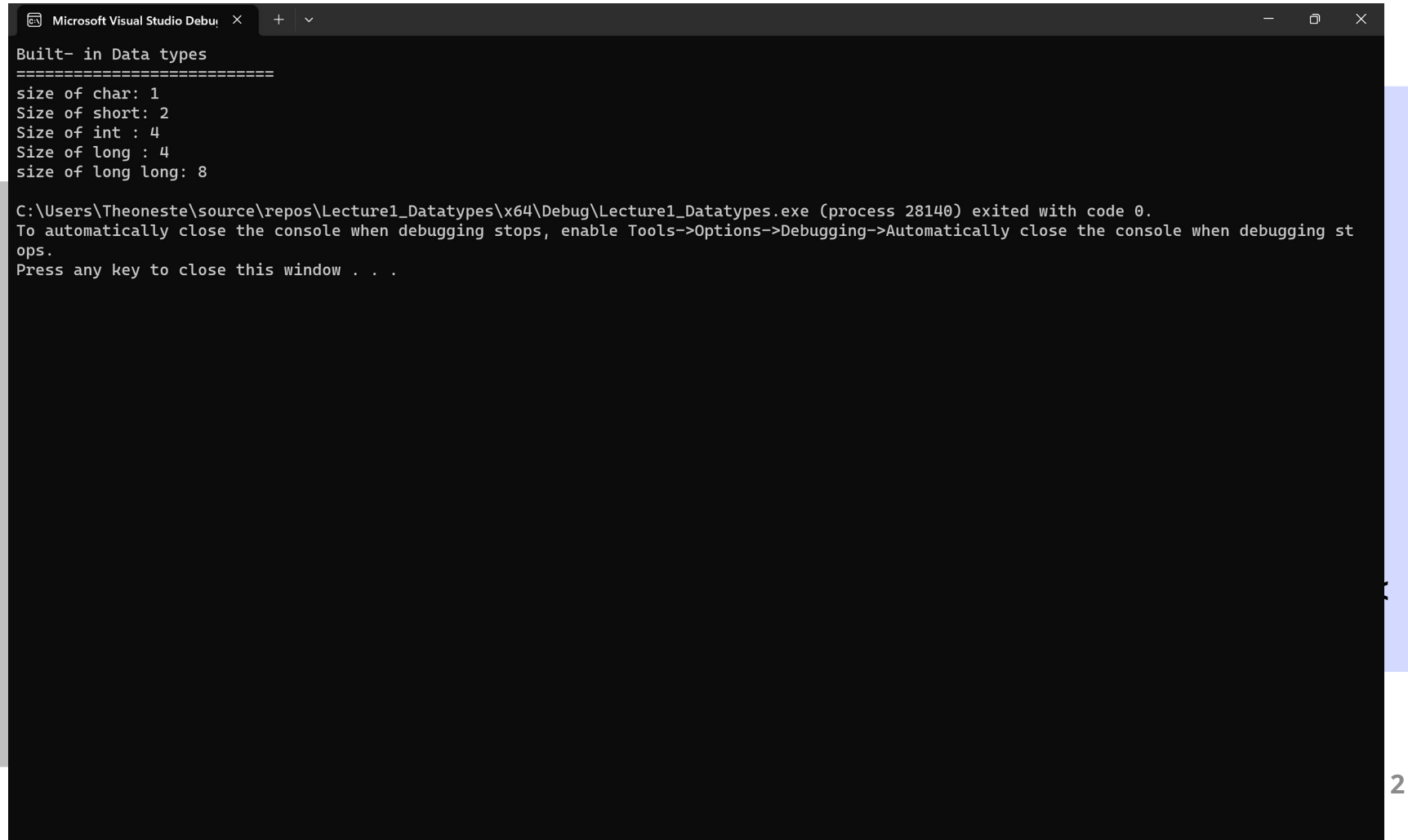
```
#include <iostream>

int main()
{
std::cout << "Built- in Data types \n";
std::cout << "============================\n";
std::cout << "size of char: " << sizeof(char) << "\n";
std::cout << "Size of short: " << sizeof(short) << "\n";
std::cout << "Size of int : " << sizeof(int) << "\n";
std::cout << "Size of long : " << sizeof(long) << "\n";
std::cout << "size of long long: "<< sizeof(long long) << "\n";
}
```

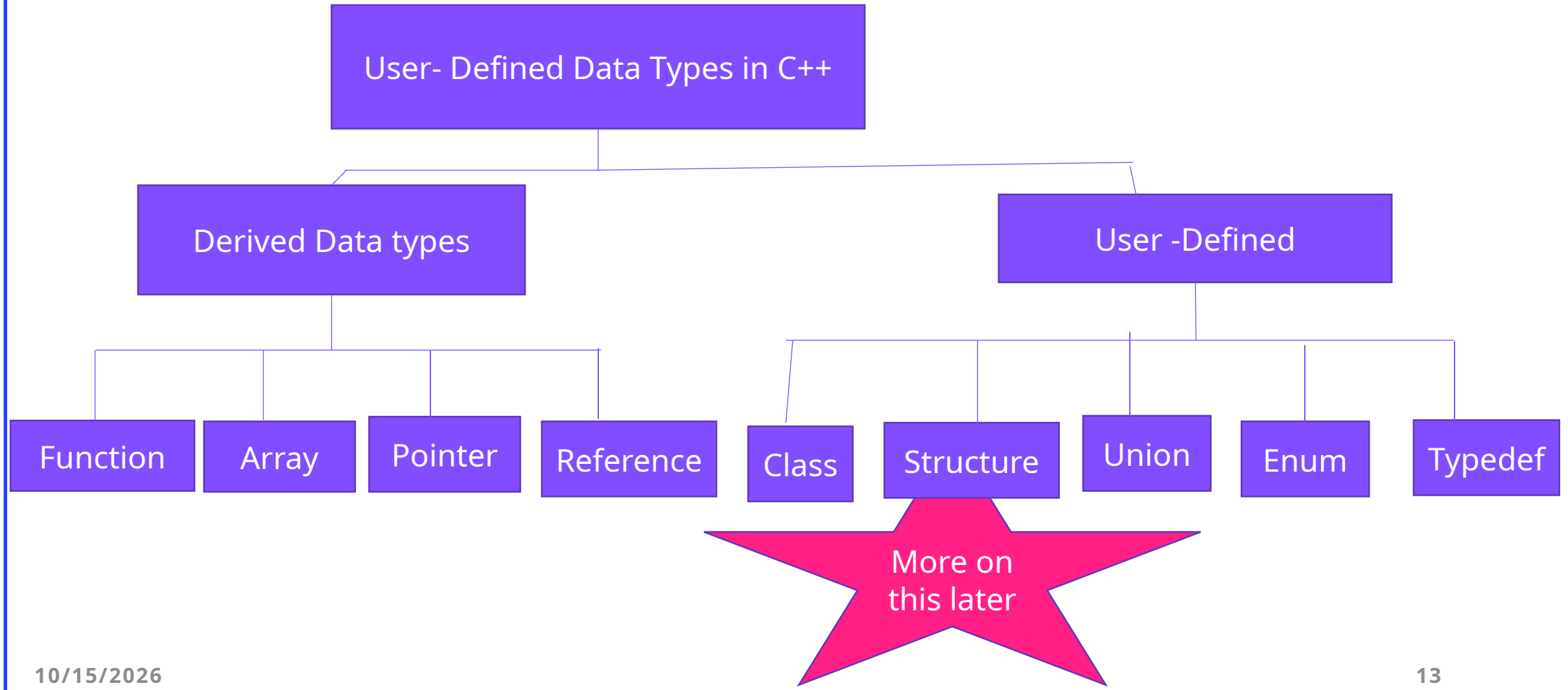
A DATA TYPE SPECIFIES THE RANGE OF VALUES A VARIABLE CAN HAVE AND THE OPERATIONS IT CAN BE INVOLVED IN.

Built-in data types are defined by the language

- Integers: (whole numbers)short, int , long, long long.
- Real numbers: (Decimal point numbers) float, double
- A Single character: char
- Boolean (true, false) : bool



C++ ALLOWS USER TO DEFINE DATA TYPES => C++ IS AN EXTENSIBLE PROGRAMMING LANGUAGE.



IN C++ EVERY VARIABLE MUST BE DECLARED AND INITIALIZED BEFORE BEING USED

```
#include <iostream>

int a = 7;

int main()
{
int b;
std::cout <<" Enter a value for
b\n";
std::cin >> b;
const int c=b;
constexpr int y = 90;

return 0;
}
```

Declaration and intialization

Declaration without initialization

The value of c will not change after assignment

Constexpr introduced in C++11 Evaluated at compile time (Whenever possible)

Qualifier (static const, volatile,

Name

Value

constexpr int y=90;

Data Type

Assignment Operato

IN C++ THE NAME OF A VARIABLE CONTAINS LETTERS, DIGITS AND UNDERSCORE. IT CANNOT START WITH A DIGIT AND IT CANNOT CONTAIN A BLANK SPACE. C++ IS CASE SENSITIVE.

Key words have a specific meaning in C++, and therefore cannot be used as identifiers (name of variables or other names entities)

The same rules apply when naming functions, classes, structures, enumerations, namespaces….

https://en.cppreference.com/w/cpp/keyword

THE SCOPE OF A VARIABLE IS THE BLOCK IN WHICH THAT VARIABLE IS ACCESSIBLE, WITHIN A PROGRAM.

```
#include <iost
int a = 7;
int main()
{
 int a = 8;
 std::cout << " a=     << a << "
 std::cout << "a= " <<  ::a << "\n
 while (a-- > 0)
  {
    static int c =
    int d = 1;
    ++c;
    ++d;
    std::cout << "c=          c << "\t" << "d= " << d << "\n";
  }
 return 0;
}
```

Global variable (Declared outside any function)

The Local Variable hides the global one (8)

Global value printed (7)

Scope resolution operator

Even though, it is legal, it is highly discouraged to use the same variable in different scopes of the same program

THE SCOPE OF A VARIABLE IS THE BLOCK IN WHICH THAT VARIABLE IS ACCESSIBLE, WITHIN A PROGRAM.

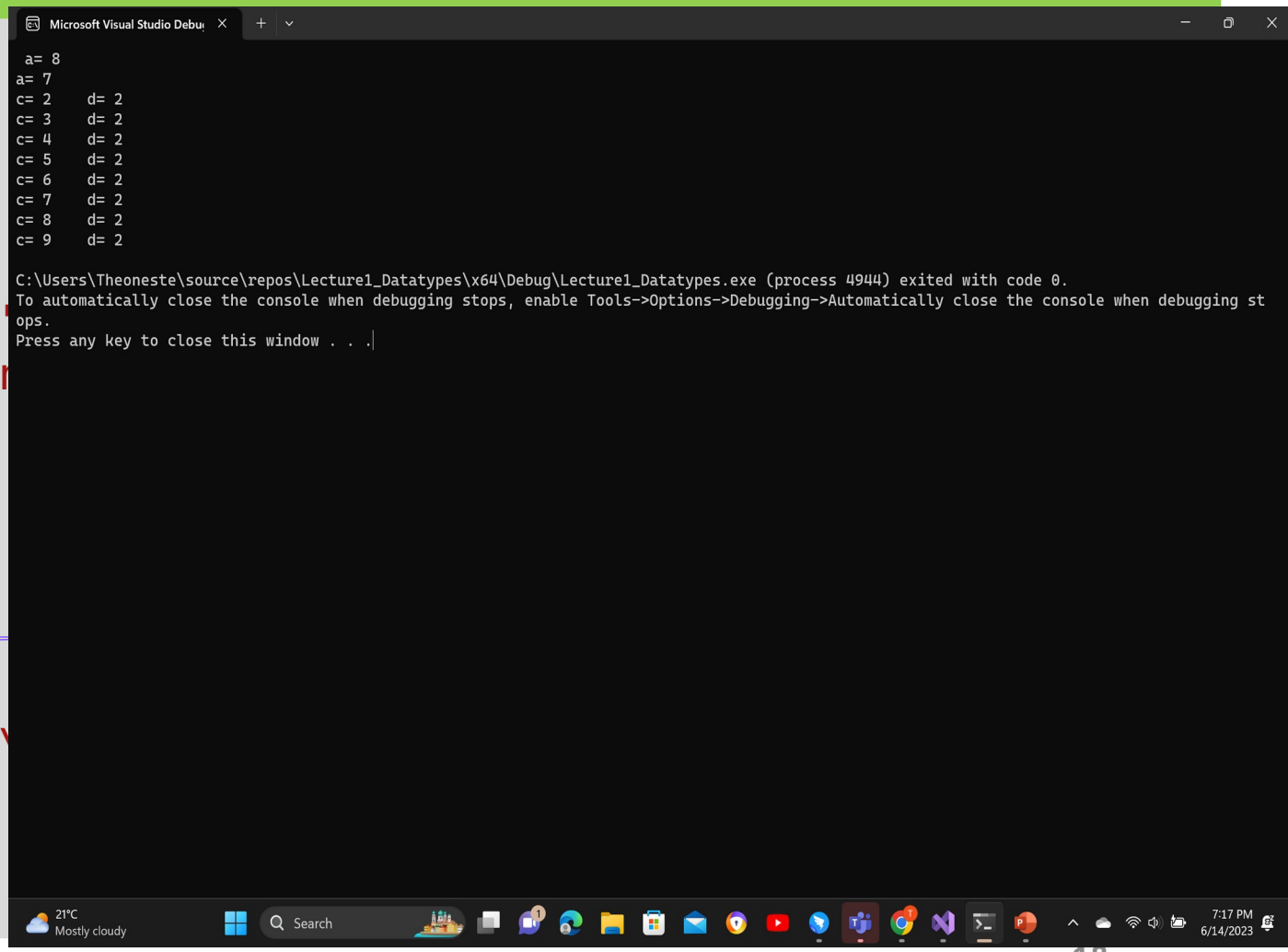
```
#include <iostream>
int a = 7;
int main()
{
 int a = 8;
 std::cout << " a=
 std::cout << "a=
 while (a-- > 0)
  {
    static int c = 1;
    int d = 1;
    ++c;
    ++d;
    std::cout << "c= " << c << "\t" << "d= " << d << "
  }
 return 0;
}
```

The variables c and d are local to the while loop. The variable c is static=> It is initialized only once. When execution leaves its scope, c is inaccessible, but retains its value.

The variable d is automatic => Every time execution leaves its scope, the variable is destroyed and recreated when execution comes back to the scope

THE SCOPE OF A VARIABLE IS THE BLOCK IN WHICH THAT VARIABLE IS ACCESSIBLE, WITHIN A PROGRAM.

```
#include <iostream>
int a = 7;
int main()
{
 int a = 8;
 std::cout << " a= " << a << "\n"
 std::cout << "a= " << ::a << "\n
 while (a-- > 0)
  {
    static int c = 1;
    int d = 1;
    ++c;
    ++d;
    std::cout << "c= " << c << "\
  }
 return 0;
}
```



THE SCOPE OF A VARIABLE IS THE BLOCK IN WHICH THAT VARIABLE IS ACCESSIBLE, WITHIN A PROGRAM.

Life Time

```
#include <iostream>
int a = 7;
int main()
{
 int a = 8;
 std::cout << " a= " << a << "\n";
 std::cout << "a= " << ::a << "\n";
 while (a-- > 0)
  {
    static int c = 1;
    int d = 1;
    ++c;
    ++d;
    std::cout << "c= " << c << "\t" << "d= " << d << "
  }
 return 0;
}
```

Global variables and live and are accessible from the point of their declaration up to the end of the program

Static variables live from their declaration up to the end of the program. They are not accessible outside of their scope

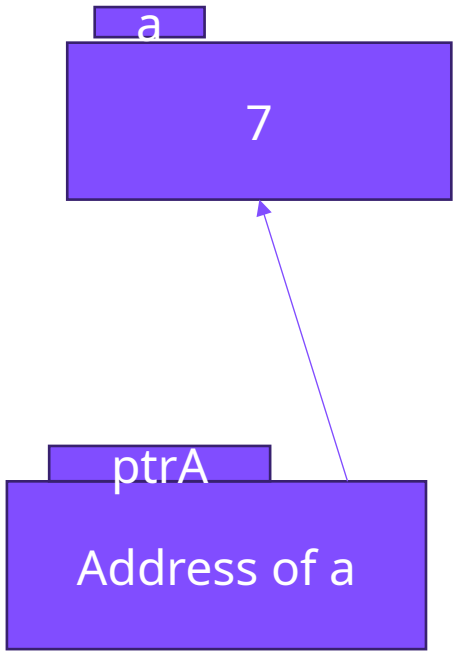
Automatic local variables exist when the program execution is in their scope

EVERY VARIABLE HAS AN ADDRESS. C AND C++ GIVE THE PROGRAMMER DIRECT ACCESS TO THE ADDRESS OF A VARIABLE USING THE ADDRESSOF OPERATOR

```
#include <iostream>

int main()
{
int a = 7;
int* ptrA = &a;
std::cout << "Value of a : " << a << "\n";
std::cout << "Address of a : " << ptrA << "\n";

return 0;
}
```





OPERATORS

C++ SUPPORTS MULTIPLE OPERATORS OPERATORS ENABLE COMPUTATION

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

ARITHMETIC OPERATORS

RELATIONAL OPERATORS

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

LOGICAL OPERATORS

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

ASSIGNMENT OPERATORS

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

INCREMENT (DECREMENT OPERATORS CAN BE USED AS PREFIX OR POSTFIX.

Conditional (Ternary Operator)

```
int c = a > b ? --a : b++;
```

Expression 1

Expression 2

Expression 3

```
#include <iostream>

int main()
{
int a = 8;
int b = 10;
int c = a > b ? --a : b++;
std::cout << "a= " << a << "\n";
std::cout << "b= " << b << "\n";
std::cout << "c= " << c << "\n";
return 0;
}
```

If expression 1 evaluates to true, then expression 2 is evaluated, otherwise expression 3 is evaluated

<
<< >>

PRECEDENCE OF OPERATORS DETERMINES THE ORDER OF OPERATIONS. WHEN OPERATORS HAVE THE SAME PRECEDENCE, ASSOCIATIVITY DETERMINES THE ORDER OF OPERATIONS.

* / %	+ -	<< >>

When in doubt use parentheses

https://en.cppreference.com/w/cpp/language/operator_precedence

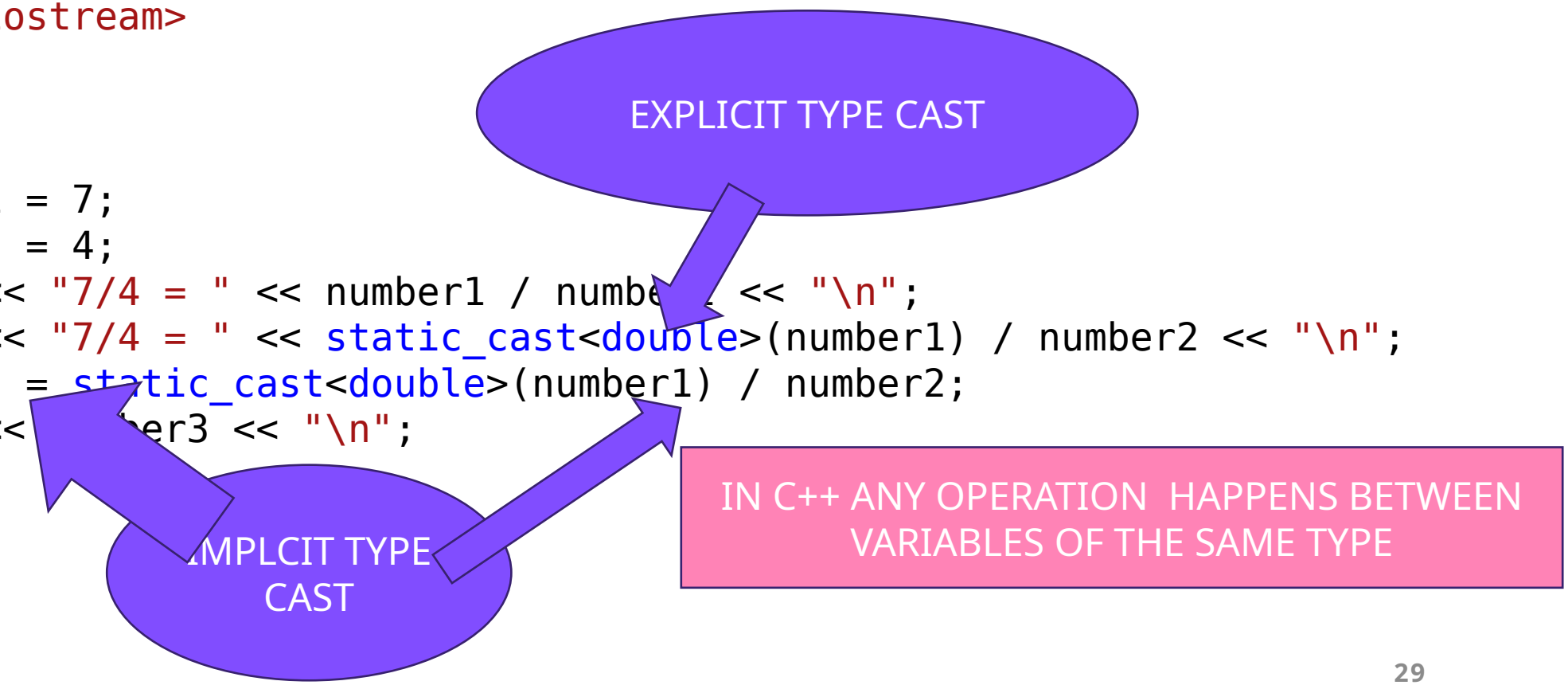
C++ HAS MANY MORE OPERATORS

- Function call ()
- Array subscript []
- sizeof operator already seen
- new and delete operators
- Bitwise operators
- Scope resolution operator already seen
- And many more…

TYPE CASTING CONSISTS IN CONVERTING A VARIABLE TO A DIFFERENT TYPE. C++ HAS A NUMBER OF TYPE CAST OPERATORS.

```
#include <iostream>

int main()
{
int number1 = 7;
int number2 = 4;
std::cout << "7/4 = " << number1 / numbe  << "\n";
std::cout << "7/4 = " << static_cast<double>(number1) / number2 << "\n";
int number3 = static_cast<double>(number1) / number2;
std::cout <<  ber3 << "\n";
return 0;
}
```





Expression is the combination of the constants, variables, and operators, which are arranged according to the syntax of C++ language and, after computation, return some values that may be a boolean, an integer, or any other data

An expression is a sequence of *operators* and their *operands*, that specifies a computation. Expression evaluation may produce a result (e.g., evaluation of `2 + 2` produces the result `4`) and may generate side-effects (e.g. evaluation of `std::printf("%d", 4)` prints the character `'4'` on the standard output). Each C++ expression is characterized

https://en.cppreference.com/w/cpp/language/expressions

and a value category.



CONTROL STRUCTURES

DECISION CONTROL STRUCTURES



DECISION CONTROL STRUCTURES ALLOW PROGRAMS TO MAKE DECISIONS BASED ON SOME CONDITION. BASED ON A CONDITION THE PROGRAM SELECTS WHAT TO DO, AMONG MANY ALTERNATIVES.

```
#include <iostream>

int main()
{
int number;
std::cout << "Enter a positive number \n";
std::cin >> number;
if (number > 0)
   std::cout << "Great you have followed  the
instruction\n";
return 0;
```



DECISION CONTROL STRUCTURES ALLOW PROGRAMS TO MAKE DECISIONS BASED ON SOME CONDITION. BASED ON A CONDITION THE PROGRAM SELECTS WHAT TO DO, AMONG MANY ALTERNATIVES.

```
#include <iostream>

int main()
{
int number;
std::cout << "Enter a positive number \n";
std::cin >> number;
if (number > 0)
    std::cout << "Great you have followed  the
instruction\n";
else
```



DECISION CONTROL STRUCTURES ALLOW PROGRAMS TO MAKE DECISIONS BASED ON SOME CONDITION. BASED ON A CONDITION THE PROGRAM SELECTS WHAT TO DO, AMONG MANY ALTERNATIVES.

```
#include <iostream>

int main()
{
int number;
std::cout << "Enter a positive number \n";
std::cin >> number;
if (number > 0)
    std::cout << "Great you have followed  the
instruction\n";
else
```



```cpp
#include <iostream>

int main()
{
 double cumulative_average;
 char letter_grade;
 std::cout << " Enter your average: \n";
 std::cin >> cumulative_average;
 if (cumulative_average >= 80)
    letter_grade = 'A’;
 else if (cumulative_average >= 70)
    letter_grade = 'B’;
 else if (cumulative_average >= 60)
    letter_grade = 'C’;
 else if (cumulative_average >= 50)
    letter_grade = 'D’;
 else
    letter_grade = 'F’;

 std::cout << "Average : " << cumulative_average << "\n";
 std::cout << "Letter grade :" << letter_grade << "\n";

 return 0;
}
```

SWITCH ….CASE STATEMENT WORKS IN A SIMILAR WAY AS IF ELSE IF….ELSE BUT WITH SOME RESTRICTIONS

```
#include <iostream>
int main()
{
int number;
std::cout << "Enter a positive integer\n";
std::cin >> number;
switch (number % 3)
{
    case 0:
    std::cout << number << " is divisible by 3\n";
    case 1:
    std::cout << "remainder is  1\n";
    case 2:
    std::cout << "remainder is 2 \n";
    default:
    std::cout << "Unkown result\n";
}
return 0;
}
```



```
#include <iostream>
int main()
{
int number;
std::cout << "Enter a positive integer\n";
std::cin >> number;
switch (number % 3)
{
   case 0:
   std::cout << number << " is divisible by 3\n";
   break;
   case 1:
   std::cout << "remainder is  1\n";
   break;
   case 2:
   std::cout << "remainder is 2 \n";
   break;
   default:
   std::cout << "Unkown result\n";
}
return 0;
}
```



CONTROL STRUCTURES

REPETITION CONTROL STRUCTURES



IN A LOOP (REPETITION CONTROL STRUCTURE) A GROUP OF STATEMENTS IS REPEATEDLY EXECUTED, UNTIL A CONDITION IS FALSE.

```
#include <iostream>
int main()
{
 int number;
 int i = 0;
 std::cout << "Enter a positive number : \n";
 std::cin >> number;
 while (i < number)
  {
    std::cout << "Hello World\n";
    i++;
  }
 std::cout << " From Kigali\n";
 return 0;
}
```

Assume number is 5. What will be the output of this program?



IN A LOOP (REPETITION CONTROL STRUCTURE) A GROUP OF STATEMENTS IS REPEATEDLY EXECUTED, UNTIL A CONDITION IS FALSE.

```
#include <iostream>
int main()
{
 int number;
 int i = 0;
 std::cout << "Enter a positive number : \n";
 std::cin >> number;
 while (i < number)
  {
    std::cout << "Hello World\n";
    i++;
  }
 std::cout << " From Kigali\n";
 return 0;
}
```

Assume number is -1. What will be the output of this program?

A WHILE IS ENTRY CONTROLLED – A DO WHILE LOOP IS EXIT CONTROLLED

While loop executes o or more times

```
#include <iostream>

int main()
{
int number;
int i = 0;
std::cout << "Enter a number : \n";
std::cin >> number;
while (i < number)
{
std::cout << "Hello World\n";
i++;
}
std::cout << " From Kigali\n";


return 0;
}
```

DO While loop executes 1 or more times

```
#include <iostream>

int main()
{
 int number;
 int i = 0;
 std::cout << "Enter a number : \n";
 std::cin >> number;
 do
  {
     std::cout << "Hello World\n";
     i++;
  }while (i < number);
std::cout << " From Kigali\n";


return 0;
}
```

A FOR LOOP HAS THREE EXPRESSIONS : INITIALIZATION , CONDITION, INCREMENT (DECREMENT)

```
#include <iostream>
int main()
{
 int number;
 std::cout << "Enter a positive number : \n";
 std::cin >> number;
 for(int i=0;i<number;++i)
   {
   std::cout << "Hello World\n";
   }
std::cout << " From Kigali\n";
return 0;
}
```

A FOR LOOP HAS THREE EXPRESSIONS : INITIALIZATION , CONDITION, INCREMENT (DECREMENT)

```
#include <iostream>
int main()
{
 int number;
 int i=0;
 std::cout << "Enter a positive number : \n";
 std::cin >> number;
 for(;i<number;++i)
    {
    std::cout << "Hello World\n";
    }
std::cout << " From Kigali\n";
return 0;
}
```

A FOR LOOP HAS THREE EXPRESSIONS : INITIALIZATION , CONDITION, INCREMENT (DECREMENT)

```
#include <iostream>
int main()
{
 int number;
 int i=0;
 std::cout << "Enter a positive number : \n";
 std::cin >> number;
 for(;i<number;)
    {
    std::cout << "Hello World\n";
    ++i;
    }
std::cout << " From Kigali\n";
return 0;
```

A FOR LOOP HAS THREE EXPRESSIONS : INITIALIZATION , CONDITION, INCREMENT (DECREMENT)

```
#include <iostream>
int main()
{
 int number;
 int i=0
 std::cout << "Enter a positive number : \n";
 std::cin >> number;
 for(; ;)
    {
     std::cout << "Hello World\n";
     ++i;
     if(i>=number)
        break;
}
```

Break statement terminates the immediately enclosing loop



```
#include<iostream>
int main()
{
int number;
std::cout << "Enter a positive number : \n";
std::cin >> number;
for(int i=0;i<number;++i)
{
 if (number % 2)
    continue;
 std::cout << number<<"\n";
}
std::cout << " From Kigali\n";
return 0;
}
```

Continue statement cuts short the current iteration and moves to the next.



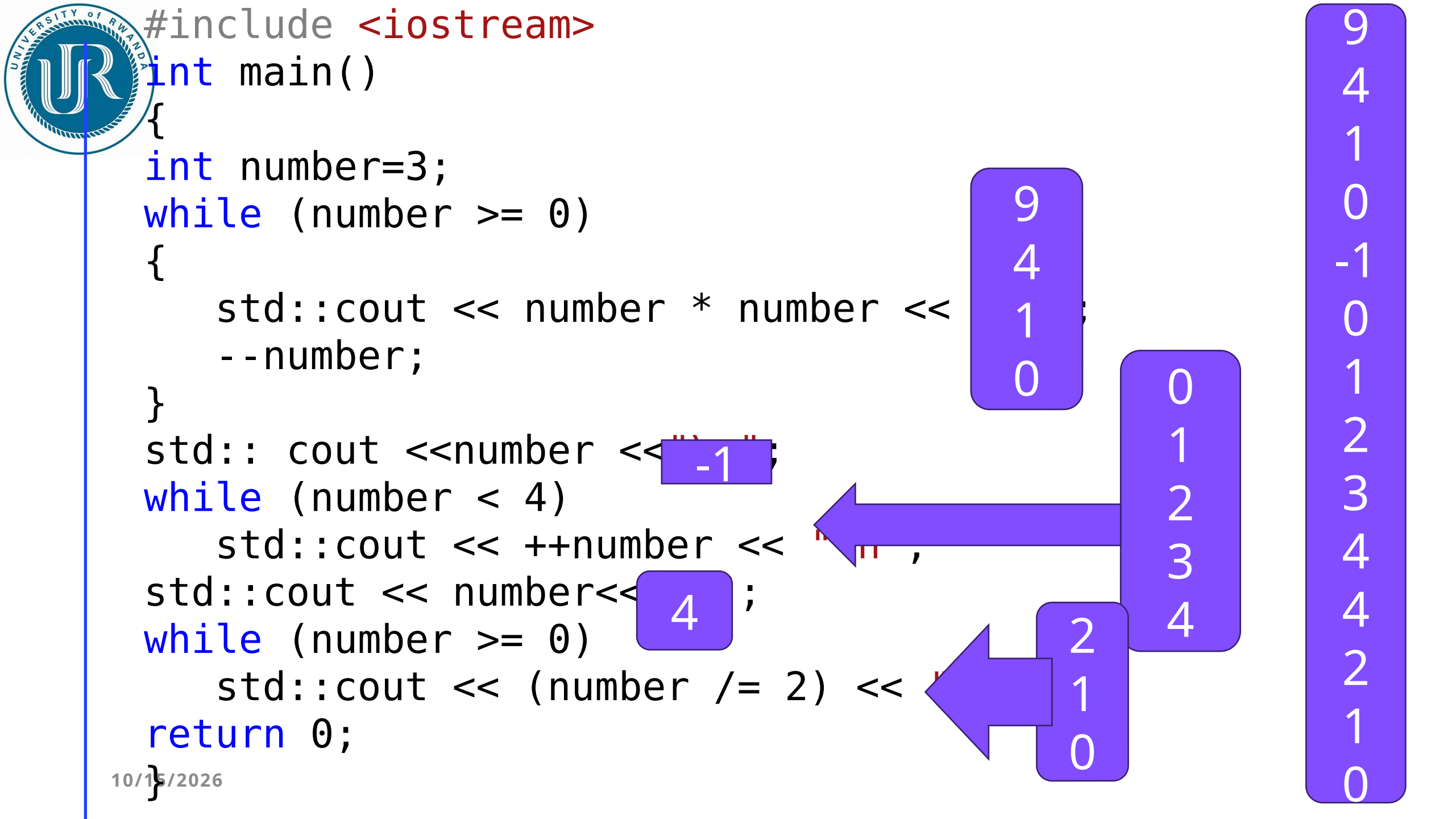
```
#include <iostream>
int main()
{
int number=3;
while (number >= 0)
{
    std::cout << number * number << "\n";
    --number;
}
std:: cout <<number <<"\n";
while (number < 4)
    std::cout << ++number << "\n";
std::cout << number<<"\n";
while (number >= 0)
    std::cout << (number /= 2) << "\n";
return 0;
}
```

WHAT IS THE OUTPUT ??



```
#include <iostream>
int main()
{
int number=3;
while (number >= 0)
{
   std::cout << number * number << "\n";
   --number;
}
std:: cout <<number <<"\n";
while (number < 4)
   std::cout << ++number << "\n";
std::cout << number<<"\n";
while (number >= 0)
   std::cout << (number /= 2) << "\n";
return 0;
}
```

```
9
4
1
0
-1
0
1
2
3
4
4
2
1
0
```


#include <iostream>
int main()
{
int number=3;
while (number >= 0)
{
std::cout << number * number <<
--number;
}
std:: cout <<number <<
while (number < 4)
std::cout << ++number <<
std::cout << number<<
while (number >= 0)
std::cout << (number /= 2) <<
return 0;
}
9
4
1
0
-1
4
0
1
2
3
4
2
1
0
9
4
1
0
-1
0
1
2
3
4
4
2
1
0



```cpp
#include <iostream>
int main()
{
int x=10;
std::cout << --x + 1 << "\n";
std::cout << x++ << "\n";

return 0;
}
```

What is the ouput??

```cpp
#include <iostream>
int main()
{
int x=1,y=2;
std::cout << x - y + 3 * 4 / 5 << "\n";

return 0;
}
```

What is the output??

```
#include <iostream>
int main()
{
int grade = 100;
if (grade > 60 and grade < 100)
    std::cout << "Success \n";
else
    std::cout << "Fail\n";
return 0;
}
```

What is the output??



```cpp
#include <iostream>
int main()
{
   int x = 1, y = 2;
   switch (y)
   {
   case 1:
   std::cout << "One\n";
   case 2:
   std::cout << "Two\n";
   default:
   std::cout << "Three\n";
}
return 0;
}
```

What is the output ??



```
#include <iostream>
int main()
{
int number_legs;
std::cout << "Enter number of legs\n";
std::cin >> number_legs;
if (number_legs == 1)
      std::cout << "Pirate\n";
else if (number_legs == 2)
      std::cout << "Bird\n";
else if (number_legs == 4)
      std::cout << "Cat\n";
else if (number_legs == 6)
      std::cout << "Spider\n";
else
      std::cout << "Idon't know who you are\n";
return 0;
}
```

Rewrite this program using switch –case instead of if –else



Rewrite this program using a while loop instead of the for loop!

```
#include <iostream>
int main()
{
int max_number;
std::cout << "Enter max number\n";
std::cin >> max_number;
for (int i = max_number; i >= 0; i--)
    std::cout << i << "\n";

return 0;
}
```

ASSIGNMENT 1 – PROBLEM 1

You own a bicycle rental business and you want to write a program that will help you determine what your customers will have to pay you for a rental. You are going to provide two integers as input. The first integer represents the starting time (hour) and the second integer represent the ending time (hour). Decimal numbers are not accepted.

(1) The starting time will always be less than the ending time. Also no rental can span more than a day. The start time will be between 0 and 23 and the ending time will be between 1 and 24.

(2) The rate per hour depends on the range as follows :

0 - 7 and 21- 24 : the rent will be 500 RWF per hour

7-14: and 19h-21: rent will be 1000 RWF per hour

14-19 : the rent will be 1500 RWF per hour

Write a C++ Program that implements the above bicycle rental program

ASSIGNMENT 1 PROBLEM 2

Mushrooms

- Agaric Jaunissant
- Amanite tue-mouche
- Cepe de bordeau
- Coprin chevelu
- Girolle
- Pied bleu

- The cepe bordeaux is the only one to have pores, the other mushrooms having gills.
- Both coprin chevelu and agaric jaunissant grow in meadows, other mushrooms grow in forests.

- The only mushrooms to have a convex cup are agaric jaunissant,amanite tue-mouches, and pied bleu.
- The only mushrooms to have a ring are agaric jaunissant,amanite tue-mouches, and coprin chevelu

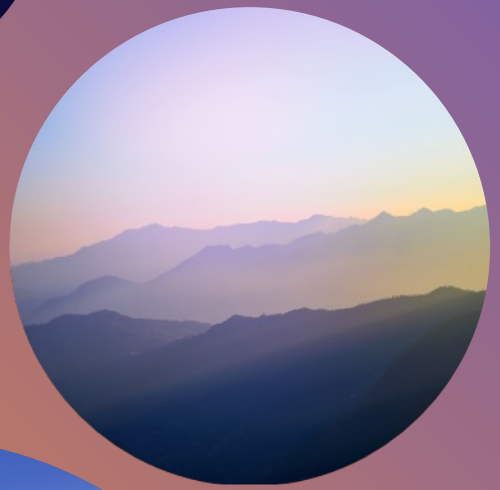
ASSIGNMENT 1 PROBLEM 2 (Cont…)

- Does your mushroom have gills ?
- Does your mushroom grow in a forest ?
- Does your mushroom have a ring?
- Does your mushroom have a convex cup?

- The User thinks of a mushroom (one of the six).
- The program asks at most three of the four questions
- The User answers truthfully by yes or no.
- Based on the user's answers, the program determines the mushroom

Write a C++ Program that implements this scenario.









THANK YOU

Presenter name
Email address
Website